Blockchain Experiment 08

Aim: To implement the Blockchain platform Ganache and deploy a simple Solidity smart contract using Metamask and Remix IDE.

Theory:

Ganache is a personal Ethereum blockchain used for testing and development. It allows developers to deploy contracts, develop decentralized applications (DApps), and run tests in a safe and deterministic environment.

Ganache provides:

- Instant mining of blocks
- Pre-funded test accounts
- Transaction history view
- Graphical interface to monitor blockchain activities

It is part of the Truffle Suite and is commonly used with Remix and Metamask for smart contract deployment.

Steps to Connecat Ganache with Metamask and Remix IDE:

1. Install Ganache:
   - Download and install Ganache from the Truffle Suite website.
   - Launch it to start a local blockchain network with test Ether accounts.
2. Connect Ganache Accounts with Metamask:
   - Open Metamask and switch to a Custom RPC Network.
   - Enter the Ganache RPC server URL (e.g., HTTP://127.0.0.1:7545).
   - Import private keys of Ganache accounts into Metamask.
3. Connect Remix IDE with Metamask:
   - Open Remix IDE in the browser.
     Choose Injected Web3 as the environment in the "Deploy & Run Transactions" tab.
   - Remix will automatically connect to the Metamask account (linked with Ganache).
4. Create a Simple Solidity Smart Contract:
   - Write a Solidity program related to your mini project (e.g., Student Record, Donation Tracker, etc.).

```
pragma solidity ^0.8.0;
    contract SimpleStorage {
      uint public data;
      function set(uint x) public {
        data = x;
      }
```

5. Compile and Deploy the Contract:
   Compile using Remix.
   - Deploy via the Metamask account connected to Ganache.
   - Approve the transaction in Metamask.
6. Check Transaction Details on Ganache:
   - View mined transactions, block details, and balances in the Ganache interface.
7. Interact with the Smart Contract:
   - Use Remix to call contract functions (set() or get()).
   - Observe the state changes and corresponding transactions on Ganache.

Outcome**:**

- Understood how to link **Metamask** with **Remix IDE** for Ethereum smart contract deployment.
- Successfully deployed and interacted with a Solidity smart contract through **Ganache**.
- Verified all transactions and balances within the Ganache environment.

Implementation:

- FarmerContract.sol:

```
// SPDX-License-Identifier: GPL-3.0
pragma solidity ^0.8.0;
contract FarmerContract {
   struct Batch {
      uint256 batchID;
      string cropName;
      uint256 quantity;
      uint256 timestamp;
      address farmer;
   }
   mapping(uint256 => Batch) public batches;
   event BatchCreated(
      uint256 batchID,
      string cropName,
      uint256 quantity,
      uint256 timestamp,
      address farmer
   );
   function addBatch(uint256 _batchID, string
memory _cropName, uint256 _quantity) public {
      require(_batchID != 0, "Invalid batch ID");
      require(batches[_batchID].batchID == 0, "Batch
already exists!");
      batches[_batchID] = Batch({
         batchID: _batchID,
         cropName: _cropName,
         quantity: _quantity,
         timestamp: block.timestamp,
         farmer: msg.sender
      });
      emit BatchCreated(_batchID, _cropName,
_quantity, block.timestamp, msg.sender);
   }
   function getBatch(uint256 _batchID) public view
returns (
      uint256,
      string memory,
      uint256,
      uint256,
      address
   ) {
      Batch memory b = batches[_batchID];
      require(b.batchID != 0, "Batch does not exist!");
      return (b.batchID, b.cropName, b.quantity,
b.timestamp, b.farmer);
   }
}
```

- DistributorContract.sol:

```
// SPDX-License-Identifier: GPL-3.0
pragma solidity ^0.8.0;
interface IFarmerContract {
   function getBatch(uint256 _batchID) external view
returns (
      uint256,
      string memory,
      uint256,
      uint256,
      address
   );
}
contract DistributorContract {
   struct ReceivedBatch {
      uint256 batchID;
      string cropName;
      uint256 quantity;
      uint256 timestamp;
      address farmer;
      address distributor;
   }
   mapping(uint256 => ReceivedBatch) public
receivedBatches;
   IFarmerContract public farmerContract;
   event BatchReceived(
      uint256 batchID,
      string cropName,
      uint256 quantity,
      address farmer,
      address distributor,
      uint256 timestamp
   );
   constructor(address _farmerContractAddress) {
      farmerContract =
IFarmerContract(_farmerContractAddress);
   }
   function receiveBatch(uint256 _batchID) public {
      (
         uint256 id,
         string memory cropName,
         uint256 quantity,
         , // ignore farmer timestamp
         address farmer
      ) = farmerContract.getBatch(_batchID);
      require(id != 0, "Batch does not exist in
FarmerContract");
      require(receivedBatches[_batchID].batchID ==
0, "Batch already received!");
      receivedBatches[_batchID] = ReceivedBatch({
         batchID: id,
         cropName: cropName,
         quantity: quantity,
         timestamp: block.timestamp,
         farmer: farmer,
```

```
        distributor: msg.sender
    });
    emit BatchReceived(id, cropName, quantity,
farmer, msg.sender, block.timestamp);
  }
  function getReceivedBatch(uint256 _batchID)
public view returns (
    uint256,
    string memory,
    uint256,
    uint256,
    address,
    address
  ) {
    ReceivedBatch memory rb =
receivedBatches[_batchID];
    require(rb.batchID != 0, "Batch not received!");
    return (
      rb.batchID,
      rb.cropName,
      rb.quantity,
      rb.timestamp,
      rb.farmer,
      rb.distributor
    );
  }
```

- RetailerContract.sol:

```
// SPDX-License-Identifier: GPL-3.0
pragma solidity ^0.8.0;
interface IDistributorContract {
  function getReceivedBatch(uint256 _batchID)
external view returns (
    uint256,
    string memory,
    uint256,
    uint256,
    address,
    address
  );
}
contract RetailerContract {
  struct RetailBatch {
    uint256 batchID;
    string cropName;
    uint256 quantity;
    uint256 timestamp;
    address farmer;
    address distributor;
    address retailer;
  }
  mapping(uint256 => RetailBatch) public
retailBatches;
  IDistributorContract public distributorContract;
  event BatchReceivedAtRetailer(
    uint256 batchID,
    string cropName,
    uint256 quantity,
    address farmer,
    address distributor,
    address retailer,
    uint256 timestamp
  );
  constructor(address _distributorContractAddress) {
    distributorContract =
IDistributorContract(_distributorContractAddress);
  }
  function receiveFromDistributor(uint256
_batchID) public {
    (
      uint256 id,
      string memory cropName,
      uint256 quantity,
      , // ignore distributor timestamp
      address farmer,
      address distributor
    ) =
distributorContract.getReceivedBatch(_batchID);
    require(id != 0, "Batch does not exist in
DistributorContract");
    require(retailBatches[_batchID].batchID == 0,
"Batch already received at Retailer!");
    retailBatches[_batchID] = RetailBatch({
      batchID: id,
      cropName: cropName,
      quantity: quantity,
      timestamp: block.timestamp,
      farmer: farmer,
      distributor: distributor,
      retailer: msg.sender
    });
    emit BatchReceivedAtRetailer(
      id,
      cropName,
      quantity,
      farmer,
      distributor,
      msg.sender,
      block.timestamp
    );
  }
  function getRetailBatch(uint256 _batchID) public
view returns (
    uint256,
    string memory,
```

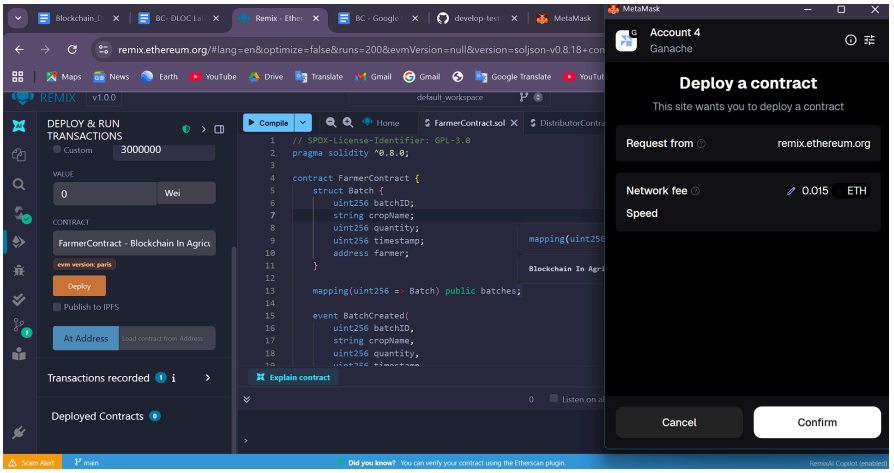
```
        uint256,
        uint256,
        address,
        address,
        address
    ) {
        RetailBatch memory rb =
retailBatches[_batchID];
        require(rb.batchID != 0, "Batch not received at
retailer!");
        return (
            rb.batchID,
            rb.cropName,
            rb.quantity,
            rb.timestamp,
            rb.farmer,
            rb.distributor,
            rb.retailer
        );
    }
```

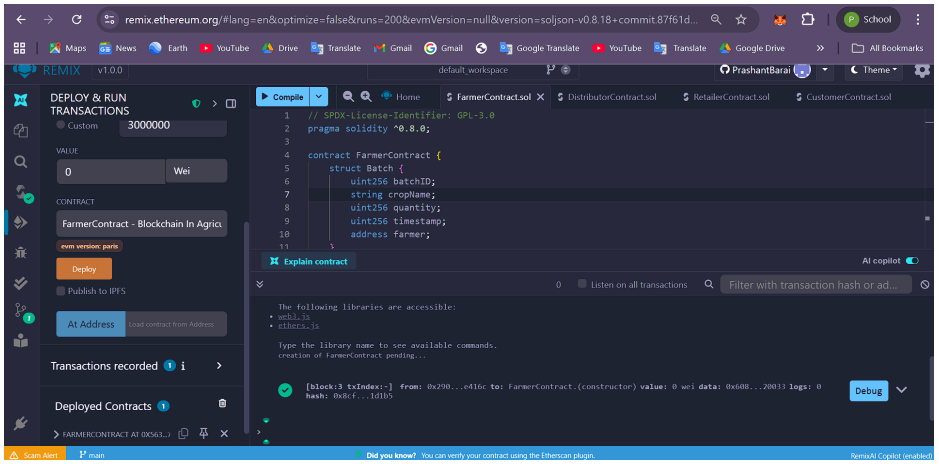
- <u>CustomerContract.sol:</u>

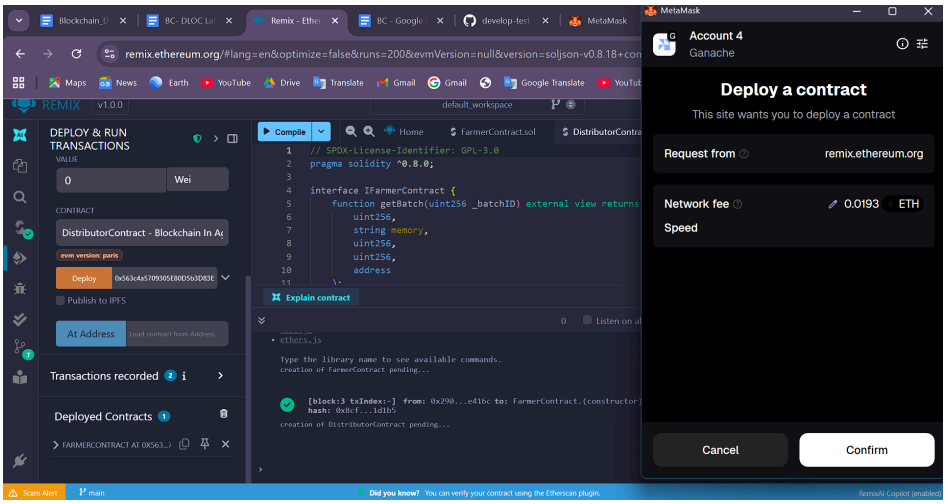
```
// SPDX-License-Identifier: GPL-3.0
pragma solidity ^0.8.0;
interface IRetailerContract {
    function getRetailBatch(uint256 _batchID)
external view returns (
        uint256,
        string memory,
        uint256,
        uint256,
        address,
        address,
        address
    );
}
contract CustomerContract {
    struct PurchasedBatch {
        uint256 batchID;
        string cropName;
        uint256 quantity;
        uint256 timestamp;
        address farmer;
        address distributor;
        address retailer;
        address customer;
    }
    mapping(uint256 => PurchasedBatch) public
purchasedBatches;
    IRetailerContract public retailerContract;
    event BatchPurchased(
        uint256 batchID,
        string cropName,
        uint256 quantity,
        address farmer,
        address distributor,
        address retailer,
        address customer,
        uint256 timestamp
    );
    constructor(address _retailerContractAddress) {
        retailerContract =
IRetailerContract(_retailerContractAddress);
    }
    function purchaseFromRetailer(uint256 _batchID)
public {
        (
            uint256 id,
            string memory cropName,
            uint256 quantity,
            , // ignore retailer timestamp
            address farmer,
            address distributor,
            address retailer
        ) = retailerContract.getRetailBatch(_batchID);
        require(id != 0, "Batch does not exist in
RetailerContract");
        require(purchasedBatches[_batchID].batchID ==
0, "Batch already purchased!");
        purchasedBatches[_batchID] =
PurchasedBatch({
            batchID: id,
            cropName: cropName,
            quantity: quantity,
            timestamp: block.timestamp,
            farmer: farmer,
            distributor: distributor,
            retailer: retailer,
            customer: msg.sender
        });
        emit BatchPurchased(
            id,
            cropName,
            quantity,
            farmer,
            distributor,
            retailer,
            msg.sender,
            block.timestamp
        );
    }
    function getPurchasedBatch(uint256 _batchID)
public view returns (
        uint256,
        string memory,
        uint256,
```

```
        uint256,
        address,
        address,
        address,
        address
    ) {
        PurchasedBatch memory pb =
purchasedBatches[_batchID];
        require(pb.batchID != 0, "Batch not
purchased!");
        return (
            pb.batchID,
            pb.cropName,
            pb.quantity,
            pb.timestamp,
            pb.farmer,
            pb.distributor,
            pb.retailer,
            pb.customer
        );
    }
}
```

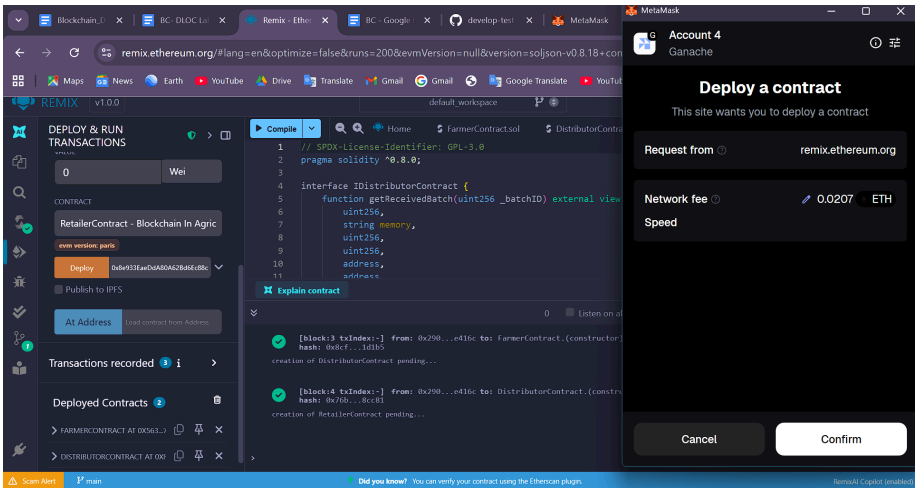
Output:

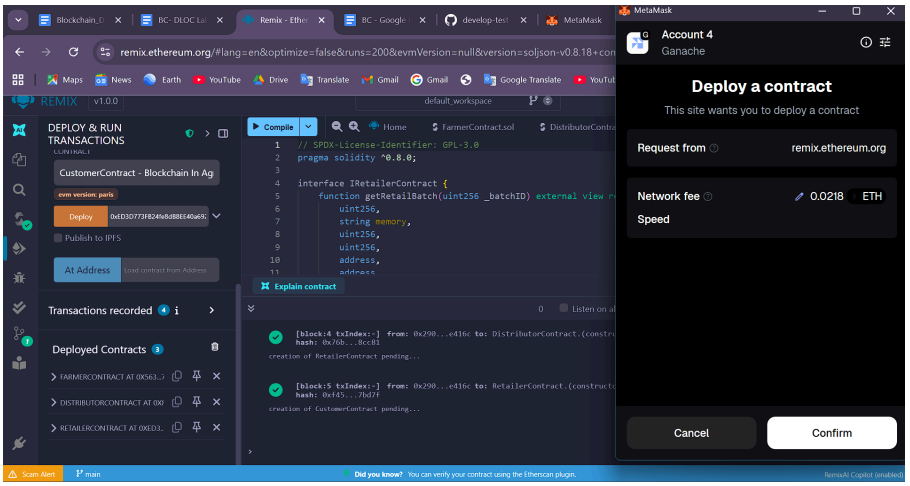


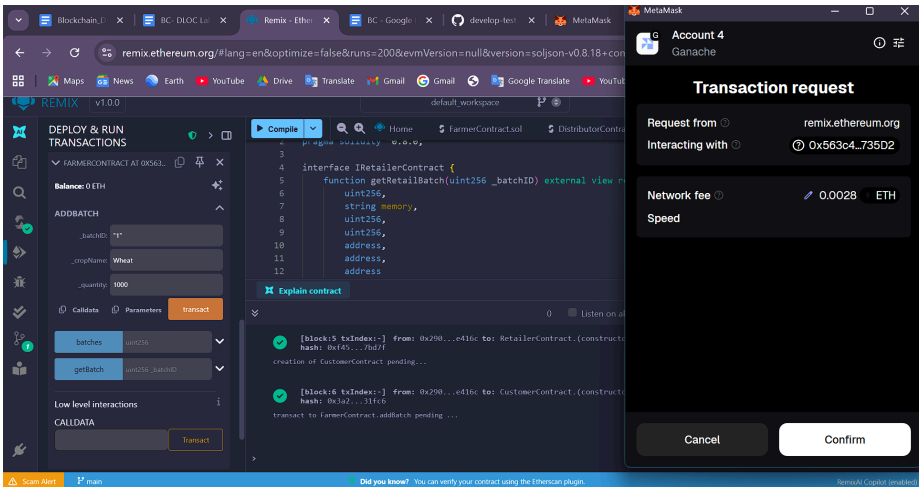


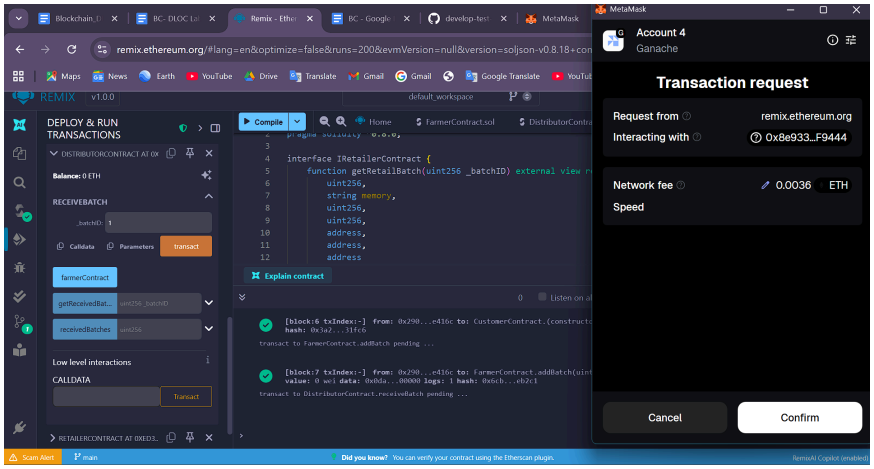


Prajyot Shinde *D20A* *Roll No.57*

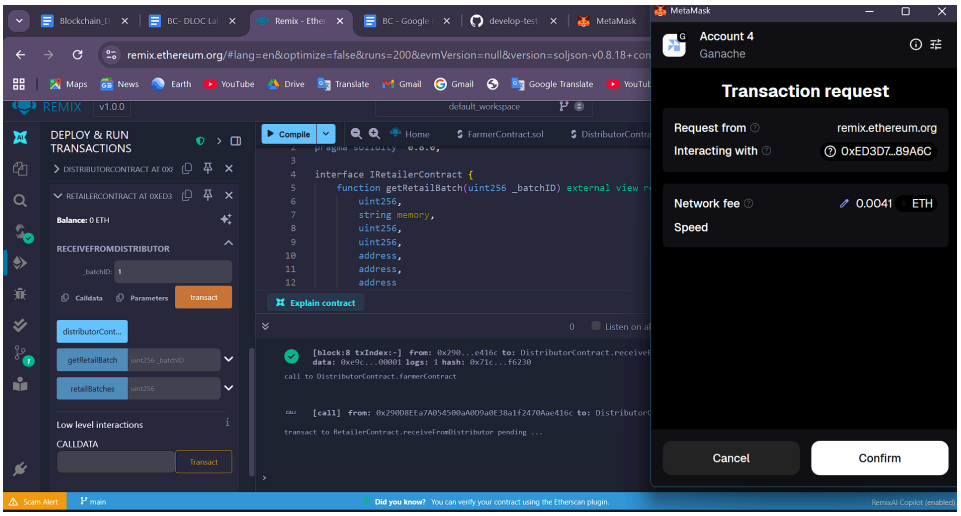


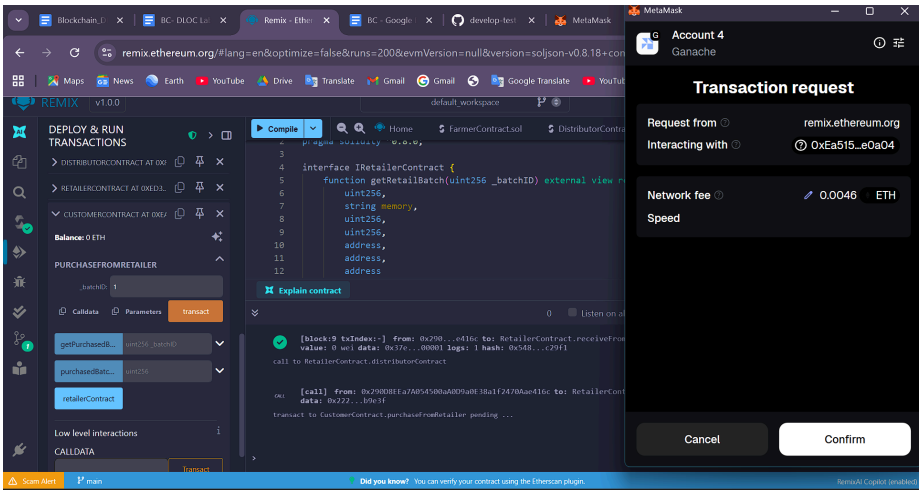


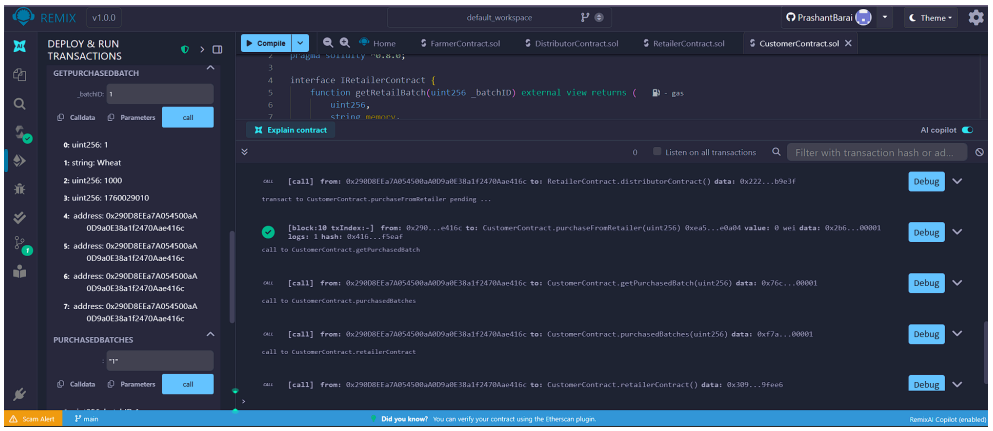


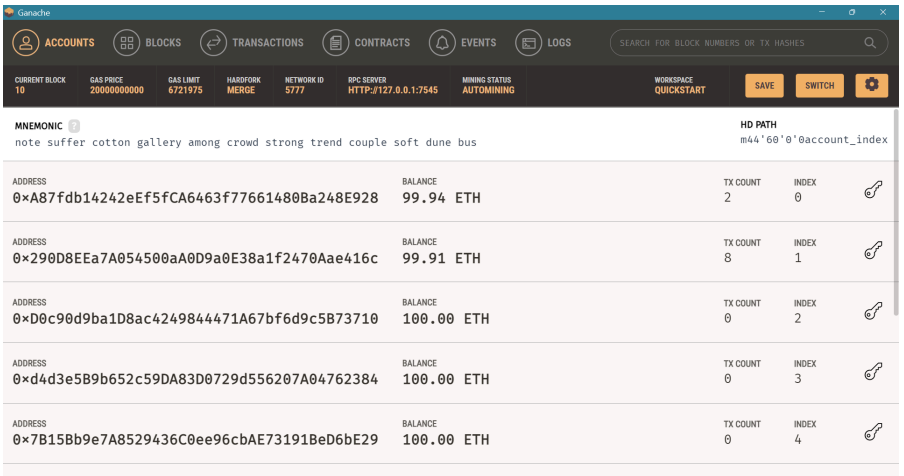


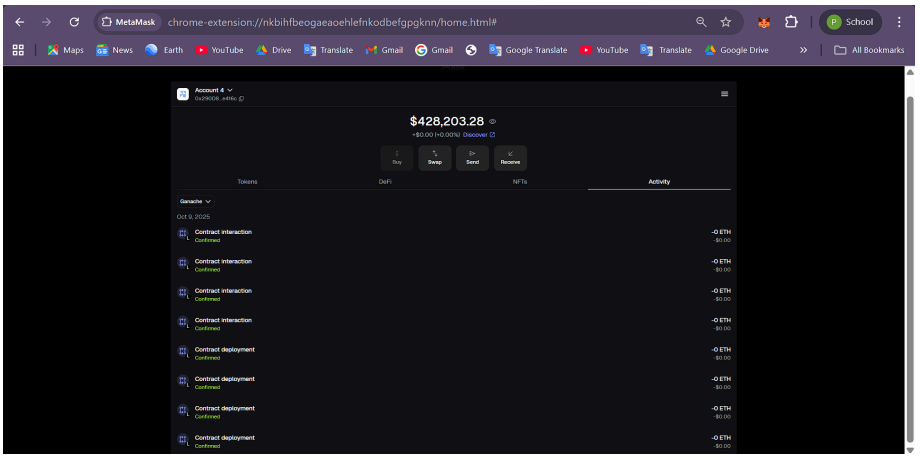
Prajyot Shinde *D20A* *Roll No.57*











Conclusion:

Ganache provides a simple yet powerful environment for simulating a local Ethereum blockchain, making it an essential tool for smart contract development. By creating a personal blockchain on a developer's machine, Ganache allows instant transaction confirmations, detailed logging, and full control over accounts, balances, and network conditions. This significantly reduces the complexity and time involved in testing, as developers are not dependent on external test networks or network delays.

When integrated with MetaMask and Remix IDE, the development workflow becomes even more efficient and seamless. Developers can deploy contracts directly from Remix to the Ganache network while using MetaMask to simulate real wallet interactions, signing transactions just as they would on a live blockchain. This combination helps in thoroughly testing contract behavior, identifying bugs, and analyzing transaction details in a controlled and risk-free setting.Additionally, Ganache's debugging tools and clear transaction history enable developers to trace errors and understand contract execution step by step, which is crucial for building secure and reliable decentralized applications. Overall, integrating Ganache with MetaMask and Remix creates a comprehensive local development ecosystem that enhances productivity, improves learning, and ensures that smart contracts are robust and well-tested before being deployed to public Ethereum networks.